



Optimizing movie cinema scheduling:

A comparison of greedy and dynamic programming algorithms

Course project final report

Assistant:

Harun Hadžo

Student:

Asja Bašović

IUS Spring 2025.

Contents

1. Introduction.....	3
2. Algorithms Description.....	4
2.1 Greedy Scheduling Algorithm	4
2.2 Dynamic Programming (Knapsack-based) Algorithm.....	4
3. Implementation Details	6
3.1 Data Structures	6
3.2 Constraints Handling	7
3.3 Helper functions.....	7
4. Theoretical Analysis	9
4.1 Time Complexity	9
4.1.1 Greedy Algorithm	9
4.1.2 DP Algorithm.....	9
4.1.3. Comparison	9
4.2 Space Complexity	10
4.2.1 Greedy Algorithm	10
4.2.3 DP Algorithm.....	10
5. Empirical Evaluation	11
6. Comparison and Practical Considerations	13
7. Conclusion	14
8. References.....	15

1. Introduction

Cinemas operate under tight scheduling constraints to maximize profit. The project objective is to design and implement a movie scheduling system that schedules movies over multiple days and screens to maximise revenue, while respecting operational constraints and improving real-world applicability. To achieve this, two separate algorithms have been developed - the greedy algorithm and the dynamic programming (DP) algorithm - in Python using appropriate libraries. The project simulates the decision-making process to build an optimized, flexible scheduling system. This project is highly relevant as it mirrors real-life optimization problems faced by cinema managers and entertainment companies. In a competitive market where cinemas must make the most of their limited screen time and cater to diverse audience expectations, a well-designed algorithm can directly influence profitability and customer satisfaction. To make the algorithms potentially useful in real-life applications, constraints such as operating hours, screen and movie formats, priorities, and revenue have been considered. In addition, when scheduling, a buffer time (the time needed to clean the venue after a showing) is considered for each movie. The project focuses on the comparison of the algorithms in terms of time and space complexity, and performance, while also commenting on which algorithm could be more useful to real-life cinemas.

2. Algorithms Description

2.1 Greedy Scheduling Algorithm

“Greedy algorithms build up a solution piece by piece, always choosing the next piece that offers the most obvious and immediate benefit” (Dasgupta et al., 2006, p.133). In the project, the main heuristic is revenue, and in case of a tie, the priority of the movie is checked (priority 3 is lowest and priority 1 is highest). This means that the immediate benefit would be the highest possible profit, and it is achieved by first sorting the movies according to expected revenue in descending order, using priority as the second sorting criterion, and then scheduling the movies in that order. The algorithm makes the best choice at the given moment without considering the overall schedule, which makes it quite fast but often suboptimal at making the highest-revenue generating schedule (GeeksforGeeks, 2024). Another potential issue with greedy algorithms is that they may overlook some movies completely if their expected revenue is low. However, in real cinemas, even less popular movies are shown, but with fewer time slots allocated to them. To solve this, the algorithm was split into two stages. In the first stage, each movie must be shown exactly once by selecting unscheduled movies that are compatible with a screen's format and sorting them by total expected revenue and priority. These are scheduled sequentially, as long as they fit within the remaining daily time window and do not overlap existing showtimes. Then, in the second stage, the algorithm fills the rest of each screen's daily schedule by greedily selecting additional compatible movies, using the same sorting logic, and scheduling them into available time slots. Buffer time is accounted for during both placement and conflict checking. The code snippet shows the code for stage 2:

```
compatible_movies = sorted( #sorting the movies in regards to expected revenue, then priority
    [m for m in movies if m["format"] in screen["supported_formats"]],
    key=lambda m: (calculate_revenue(m, ticket_prices), -m["priority"]),
    reverse=True)
for movie in compatible_movies:
    duration_with_buffer = movie["duration"] + 25
    if remaining_time >= duration_with_buffer:
        start_time = 720 - remaining_time
        if is_slot_free(screen_schedule, start_time, movie["duration"]):
            screen_schedule.append({
                "start_time": start_time,
                "end_time": start_time + movie["duration"],
                "movie": movie["title"],
                "revenue": calculate_revenue(movie, ticket_prices),
                "must": 0
            })
            remaining_time -= duration_with_buffer
```

2.2 Dynamic Programming (Knapsack-based) Algorithm

The dynamic programming algorithm used in this project is based on the 0/1 Knapsack optimization problem. In the knapsack problem, one aims to select a subset of items-each with a value and weight-to

maximize the total value without exceeding the weight limit of the knapsack (Al Etawi & Aburomman, 2020, p.1). Hence, each movie has a “value” which is its expected revenue and a “weight” which is its total screening time including a 25-minute buffer, and the goal is to maximize revenue without exceeding the 720-minute operating window of a cinema screen per day. The DP scheduler is applied independently to each screen on each day. For each compatible movie (a movie whose format matches the format of the screen currently making the schedule for), it checks whether scheduling it at time t (considering its duration and buffer) would increase the total revenue. If so, it updates the DP table and appends the movie to the list of scheduled entries, making sure there are no overlaps. As with the greedy algorithm, to meet the constraint of showing each movie at least once during the time covered in the schedule, the dynamic programming approach runs in two phases. Stage 1 ensures that every movie is scheduled at least once by prioritizing unscheduled movies based on priority and revenue, and placing them in the earliest available time slots if format-compatible. Stage 2 uses a 0/1 knapsack-like DP algorithm to maximize revenue for the remaining time. A one-dimensional DP table is built for each screen and day, where each entry at time t stores the best cumulative revenue and corresponding movie list that can be scheduled in t minutes. After building the table, the selected movies are added to the screen's schedule in available slots, ensuring no overlaps. This structure balances mandatory movie coverage with revenue optimization. Although dynamic programming is more computationally intensive than the greedy approach (Al Etawi & Aburomman, 2020, p.10), it has the advantage of exploring many possible combinations of movie schedules and often results in higher overall revenue. The code snippet shows the code for stage 2:

```
dp = [{"revenue": 0, "movies": []} for _ in range(721)]
for t in range(721):
    for movie in [m for m in movies if m["format"] in screen["supported_formats"]]:
        duration_with_buffer = movie["duration"] + 25
        if t >= duration_with_buffer:
            revenue = calculate_revenue(movie, ticket_prices)
            if dp[t - duration_with_buffer]["revenue"] + revenue > dp[t]["revenue"]:
                if is_slot_free(screen_schedule, t - duration_with_buffer, movie["duration"]):
                    dp[t] = {
                        "revenue": dp[t - duration_with_buffer]["revenue"] + revenue,
                        "movies": dp[t - duration_with_buffer]["movies"] + [movie]
                    }
for movie in dp[720]["movies"]:
    for t in range(0, 721 - movie["duration"]):
        if is_slot_free(screen_schedule, t, movie["duration"]):
            screen_schedule.append({
                "start_time": t,
                "end_time": t + movie["duration"],
                "movie": movie["title"],
                "revenue": calculate_revenue(movie, ticket_prices),
                "must": 0
            })
break
```

3. Implementation Details

3.1 Data Structures

For the algorithms to work, information about the movies to be shown and the screens available is needed. Hence, two main dictionaries were used with dummy variables for testing. The movie information includes movie ID, movie title, duration of the movie, priority level (3 is lowest and 1 is highest), expected audience (number of expected people for one showing), and screening format of the movie (2D, 3D, 4DX, IMAX). Regarding the screen information, screen ID, capacity of the screen room (an integer), and supported formats of the screen (2D, 3D, 4DX, IMAX) are stored. Additionally, in order to calculate the expected revenue of a movie the price of the ticket must be considered. Depending on the format of the movie, the price of the ticket will vary (ex. ticket for a 2D movie is \$10, while it is \$18 for a 4DX movie). Therefore, these pricings need to be stored as well. Below are the sample dictionaries used in testing the algorithms:

```
movies = [
    {"id": 1, "title": "Avatar 2", "duration": 180, "format": "3D", "audience_size": 50, "priority": 1},
    {"id": 2, "title": "Oppenheimer", "duration": 150, "format": "2D", "audience_size": 40, "priority": 2},
    {"id": 3, "title": "Barbie", "duration": 120, "format": "4DX", "audience_size": 30, "priority": 1},
    ...]
screens = [
    {"id": 1, "capacity": 100, "supported_formats": ["2D", "3D"]},
    {"id": 2, "capacity": 80, "supported_formats": ["4DX", "IMAX"]},
    ...]
ticket_prices = {"2D": 10, "3D": 15, "4DX": 20, "IMAX": 18}
```

The developed schedule is also stored as a dictionary, and it is structured as follows: the scheduled movies can be seen for each screen on each day across the specified time period. For each movie, it can be seen what the start and end time is, movie name, expected revenue and if the scheduling was forced to fulfil the “each movie must be scheduled at least once” constraint. An example schedule:

```
=== DP/Knapsack Scheduler ===
{
  "day_1": {
    "screen_1": [
      {
        "start_time": 0,
        "end_time": 100,
        "movie": "Mean Girls",
        "revenue": 375,
        "must": 1
      },
      {
        "start_time": 125,
        "end_time": 225,
        "movie": "Mean Girls",
        "revenue": 250,
        "must": 1
      },
      ...],
    "screen_2": [
```

```

    {
      "start_time": 0,
      "end_time": 170,
      "movie": "Interstellar",
      "revenue": 450,
      "must": 1
    }, ... ],
  },
  "day_2": {
    "screen_1": [
      {
        "start_time": 0,
        "end_time": 100,
        "movie": "Mean Girls",
        "revenue": 375,
        "must": 0
      },
      {
        "start_time": 125,
        "end_time": 285,
        "movie": "Dune",
        "revenue": 525,
        "must": 0
      }, ... ]
    }
  }
}

```

3.2 Constraints Handling

As mentioned, the project incorporates several real-world constraints to ensure that the generated movie schedules are usable and realistic. The first major constraint is the operating hours of the cinema. Each screen operates from 12:00 PM to 12:00 AM, providing a daily scheduling limit of 720 minutes. To model realistic transitions between screenings, each movie includes a fixed buffer time of 25 minutes (15 minutes for advertisements before the film and 10 minutes for cleaning). This buffer is added to the movie's duration when checking whether it can fit into the remaining time on a given screen. Another constraint is format compatibility, meaning that not all screens support all formats (e.g., 2D, 3D), so each movie is only considered for screens that support its designated format. To ensure completeness, a constraint requiring that each movie be shown at least once in the schedule is made. This is enforced by splitting the scheduling into two phases as described previously. The algorithm ensures no overlaps by using the `is_slot_free` function. Lastly, revenue is calculated by multiplying each movie's audience size by the ticket price for its format, which differs across screening types. These constraints collectively ensure that the schedule reflects realistic cinema operations while allowing the algorithms to function effectively.

3.3 Helper functions

To assist the functioning of the main algorithms, two helper functions have been implemented. The function `calculate_revenue` will calculate the unique revenue for each movie depending on the expected audience size for the showing and the ticket price for the format the movie is in. The function `is_slot_free` is used to check if the slot to schedule a movie is potentially taken or free. If the proposed movie ends before another movie starts, or starts after it ends (including the buffer), then the slot is free. If the intervals overlap, the

function returns False, indicating the slot is not available. Otherwise, the function returns True, meaning it can be safely added to the schedule. Codes for each function can be seen below:

```
def calculate_revenue(movie: Dict, ticket_prices: Dict) -> int:
    return movie["audience_size"] * ticket_prices[movie["format"]]

def is_slot_free(schedule: List[Dict], start_time: int, duration: int) -> bool:
    end_time = start_time + duration
    for scheduled_movie in schedule:
        scheduled_start = scheduled_movie["start_time"]
        scheduled_end = scheduled_movie["end_time"] + 25 # Buffer
        if not (end_time <= scheduled_start or start_time >= scheduled_end):
            return False
    return True
```


4. Theoretical Analysis

4.1 Time Complexity

4.1.1 Greedy Algorithm

The greedy scheduler attempts to maximize revenue each day by first ensuring all movies are shown at least once, and then filling remaining time slots on each screen with the most profitable compatible movies. The algorithm iterates over each day (D) and each screen (S) and, for each screen, it sorts movies by revenue efficiency, and then tries to place each movie into the schedule while checking for slot availability. The slot-checking procedure that ensures no time overlap involves scanning through previously scheduled movies for the screen, which is bounded by the number of time slots (N) which is the number of possible movie placements per day on that screen. In the worst case, this could require up to $M \cdot N$ checks per screen, where M is the number of movies, and it takes $M \cdot \log M$ time to sort the movies in the needed order. Therefore, the theoretical time complexity is: $O(D \cdot S \cdot (M \log M + M \cdot N))$, where D = number of days, S = number of screens, M = number of movies, and N = number of scheduled time slots per screen per day. However, in practice, N is small, as the theater day is 720 minutes long and each movie takes 120 to 200 minutes plus a buffer, each screen can only accommodate a handful of movies per day. Because N does not grow with the size of the input, it can be treated as a constant. Thus, the time complexity simplifies to: $O(D \cdot S \cdot M \log M)$.

4.1.2 DP Algorithm

The DP-based scheduler constructs an optimal set of movies for each screen and day using a 1D DP approach. For each screen (S) and day (D), the algorithm builds a DP table of size $T = 720$, representing each possible minute of the 12-hour operating window. At each time unit t, it considers every movie (M) which then sorted ($M \cdot \log M$), that fits within the remaining time and is compatible with the screen. For each such candidate, it verifies whether the movie can be scheduled at that time by checking for slot conflicts. The conflict check involves scanning previously scheduled movies on the screen, up to N entries (i.e., the number of movies already scheduled on that screen that day). This leads to $O(M \cdot N)$ and it checks this for each minute (T), giving a per-screen per-day complexity of: $O(M \cdot \log M + T \cdot M \cdot N)$. Across all D days and S screens, the complete time complexity becomes: $O(D \cdot S \cdot (M \cdot \log M + T \cdot M \cdot N))$. However, as in the greedy algorithm, T (720) and N (typically 4 to 6) are both small constants in practice, and as such, both T and N can be treated as constants. This allows us to simplify the time complexity to: $O(D \cdot S \cdot M \cdot \log M)$.

4.1.3. Comparison

While both algorithms have similar complexity under simplified assumptions, the greedy algorithm is significantly faster in practice due to its simpler operations. It avoids scanning across time windows and instead relies on sorted selection and direct slot feasibility checks. In contrast, the dynamic programming

approach must consider all possible time points and combinations, resulting in more overhead. Therefore, greedy scheduling is better suited for real-time or large-scale scenarios, while dynamic programming offers better revenue optimization at the cost of higher computation.

4.2 Space Complexity

4.2.1 Greedy Algorithm

The greedy scheduler maintains a schedule dictionary that stores all scheduled movie entries for each screen on each day. Each entry contains metadata such as start time, end time, movie title, and revenue. The amount of space required depends primarily on the number of scheduled movies. Each scheduled movie entry is stored as a dictionary, so the total space used by the schedule is proportional to $O(D*S*N)$ (D = number of days, S = number of screens, N = maximum number of scheduled entries per screen per day). Since the greedy algorithm does not build any data structures beyond this schedule and a few filtered lists of movies (at most size M), its total space complexity is: $O(D*S*N + M)$. As N is bounded and small due to the 12-hour screening limit per day, this simplifies to: $O(D*S)$. Thus, the greedy algorithm is quite memory efficient and scales well even with a moderately large number of movies.

4.2.3 DP Algorithm

The DP algorithm maintains the same schedule structure as the greedy algorithm but requires additional memory to store a DP table for each screen on each day. Each DP table has a fixed length of $T+1$, where $T=720$ represents the number of minutes in a day. At every index in the table, the algorithm stores the cumulative revenue achievable within that amount of time, and the corresponding list of movies that contribute to that revenue. In the worst case, each entry may store up to N movies, where N is the number of time slots that can fit into a single day. This results in a per-screen per-day space complexity of approximately $O(T*N)$. The algorithm also stores the final schedule itself which contributes another $O(N)$ entries per screen per day. Therefore, the total space usage across all days and screens is $O(D*S*T*N + D*S*N)$, or simplified $O(D*S*T*N)$, where D is the number of days and S is the number of screens. If we treat both T and N as constants, this simplifies to $O(D*S)$.

4.2.3 Comparison

Although both algorithms simplify to $O(D*S)$ space under constant assumptions, the dynamic programming approach uses significantly more memory in practice due to the allocation of a DP table for each screen and day, which tracks revenue and movie combinations for all 720 time steps. In contrast, the greedy algorithm only tracks the final schedule and uses lightweight filtering and sorting operations.

5. Empirical Evaluation

Empirical analysis was conducted of the runtime and memory usage of both algorithms using synthetic data. To achieve this Python's tracemalloc module was used to measure peak memory usage and time for runtime, and the results were visualized using matplotlib. The experiments varied one parameter at a time - number of movies, screens, or days - while keeping the other two fixed, to isolate the impact of each factor on runtime and memory usage. The number of movies tested ranged from 10 to 50, screens from 2 to 6, and days from 1 to 10. The greedy algorithm consistently outperformed the dynamic programming solution in both time and space usage. This aligns with the theoretical complexities discussed earlier. As seen in Figures 1, 2, and 3, each increase of the number of movies, days, and screens, respectively, negatively impacted the performance of the DP algorithm while the greedy scheduler remained mainly unaffected. It can be observed that the greedy algorithm scales better than the DP one, making it more suitable for real-life use.

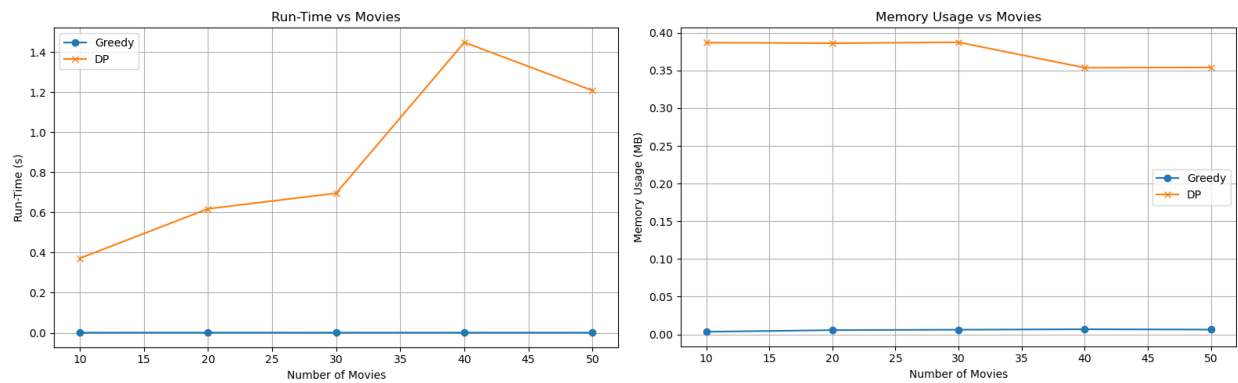


Figure 1, Run-time and memory usage when varying the number of movies

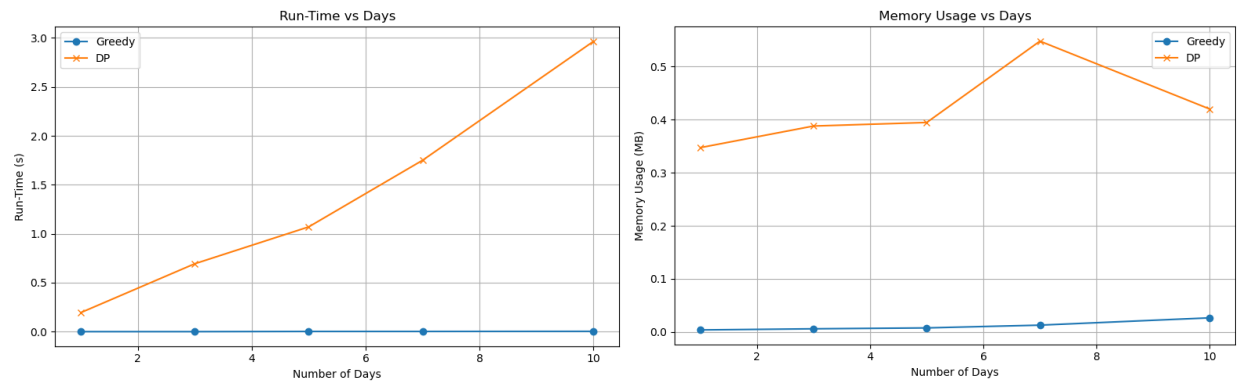


Figure 2, Run-time and memory usage when varying the number of days

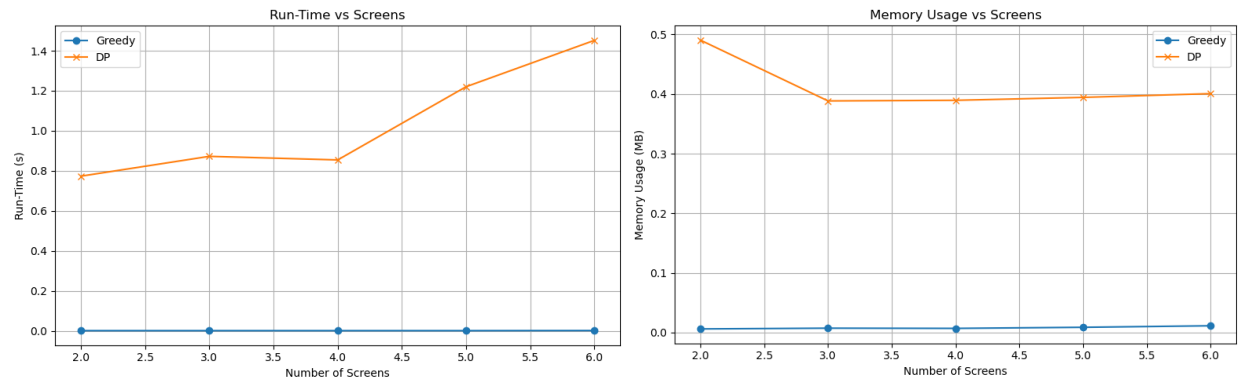


Figure 3, Run-time and memory usage when varying the number of screens

6. Comparison and Practical Considerations

The comparison between the dynamic programming and greedy scheduling algorithms reveals fundamental trade-offs between theoretical optimization and practical viability for real-world cinema operations. While both algorithms successfully schedule all mandatory showings, their approaches differ significantly in ways that impact real-world usability. The DP scheduler demonstrates perfect revenue optimization by ruthlessly exploiting the most profitable films sometimes scheduling the same film 3-4 times daily on a single screen and repeating it for all days. This occurs because the algorithm maximizes the revenue without considering audience experience. While this approach delivers theoretically perfect revenue, it creates several practical problems such as audience frustration from lack of variety. In contrast, the greedy scheduler produces more balanced schedules that better reflect real cinema needs. Its output shows higher variety across a single screen and across different screens while still respecting all mandatory showings and buffer requirements. This variety stems from the algorithm's local optimization approach, which makes compromises at each scheduling decision point. The choice between algorithms depends on what the cinema's priority is: pursuing perfect mathematical optimization or operational practicality. While the DP solution generates more revenue, the Greedy algorithm produces more sustainable, audience-friendly schedules that maintain revenue that is still very high while avoiding the customer experience pitfalls of extreme repetition. For most theaters, this balance makes the greedy approach the superior choice, but the DP algorithm could be further altered to give a more balanced schedule. For example, a new constraint on the maximum showings of a single movie in a day could be added but this would lead to suboptimal results. Therefore, further altering the DP algorithm, which can be complex, may not be needed as the greedy algorithm already meets the requirements without any additional configuration while also being faster and using less memory space.

7. Conclusion

The project explored the application of greedy and dynamic programming (DP) algorithms to the problem of cinema movie scheduling, intending to maximize revenue under realistic operational constraints. Both approaches successfully generated feasible schedules that adhered to format compatibility, operating hours, buffer times, and the requirement that every movie be shown at least once over a specified period. Empirical analysis confirmed the theoretical expectations that the greedy algorithm consistently outperformed the DP approach in terms of runtime and memory usage, while the DP algorithm often produced schedules with slightly higher total revenue due to its ability to explore more combinations. Choosing which algorithm to use depends on context. The greedy algorithm is well-suited for scenarios where execution speed, simplicity, and responsiveness are essential—such as real-time scheduling or systems with limited computational resources. On the other hand, the DP algorithm is preferable when optimal revenue is a priority and longer runtime is not an issue. Despite their effectiveness, both algorithms have limitations. The current implementations do not account for fairness across genres or popularity tiers, nor do they incorporate real-time audience feedback or staff logistics. Future improvements could explore probabilistic demand modeling, dynamic ticket pricing, and schedule personalization based on viewer preferences. Overall, the project demonstrates how classical algorithms can be adapted to solve real-world scheduling problems, highlighting the trade-offs between speed and optimality, and paving the way for further exploration into intelligent, adaptive scheduling systems.

8. References

- Al Etawi, N., & Aburomman, F. (2020). *0/1 Knapsack Problem: Greedy Vs. Dynamic Programming*. International Journal of Advanced Engineering and Management Research. https://www.ijaemr.com/uploads/pdf/archivepdf/2020/IJAEMR_393.pdf
- Dasgupta, S., Papadimitriou, C., & Vazirani, U. (2006). *Algorithms*. McGraw-Hill Education <http://algorithmics.lsi.upc.edu/docs/Dasgupta-Papadimitriou-Vazirani.pdf>
- GeeksforGeeks. (2024, December 12). *Applications, Advantages and Disadvantages of Greedy Algorithms*. GeeksforGeeks. <https://www.geeksforgeeks.org/applications-advantages-and-disadvantages-of-greedy-algorithms/>